CSA1311 - Theory of Computation

S. Srivally
192472311

Assignment-2

(1) Write closure properties of regular languages with respect to complement, Kleene star, reversal and substitution.

Ans:

A language is said to be closed under an operation to a regular language produces another regular language.

The regular languages have the following closure properties:

1. Closure under Complement

If L is a regular language over an alphabet $\Sigma$, then its complement is also regular.

The complement of L contains all strings over $\Sigma$ that do not belong to L.

Mathematically, $\overline{L} = \Sigma^* - L$

Example: Let, $L = \{a, ab\}$

Then its complement contains all strings over $\{a, b\}$ except a and ab.

Explanation: A DFA can be constructed for the complement by interchanging its final and non-final states.

Therefore, regular languages are closed under complement.

2. Closure under Kleene Star

If L is a regular language, then its kleene star $L^*$ is also regular.

Kleene star represents zero or more repetitions of strings belonging to L.

Mathematically, $L^* = \bigcup_{i=0}^{\infty} L^i$

Example:

Let, $L = \{ab\}$

Then, $L^* = \{\varepsilon, ab, abab, ababab, \ldots\}$

Here, $\varepsilon$ represents the empty string.

Explanation: An $\varepsilon$-NFA can be constructed by adding $\varepsilon$-transitions to allow zero or more repetitions. Therefore, regular languages are closed under Kleene star.

3. Closure under Reversal

If $L$ is also regular language, then its reversal $L^R$ is also regular. Reversal means reading every string in the opposite direction.

Mathematically, $\boxed{L^R = \{w^R \mid w \in L\}}$

Example:

Let, $L = \{ab, abc\}$

then, $L^R = \{ba, cba\}$

Explanation: The reversal of a finite automation can be constructed by reversing all transitions, making the original final states initial states and the original initial state a final state.

Therefore, regular languages are closed under reversal.

4. Closure under Substitution

If $L$ is a regular language and each symbol in its alphabet is replaced by a regular language, the resulting language is a regular.

Example: Let, $L = \{ab\}$

Suppose the substitution is:

$a \rightarrow \{0, 1\}$

$b \rightarrow \{11\}$

Then, $L' = \{011, 111\}$

Since the substituted languages are regular, the resulting language is also regular.

Conclusion:- Regular languages are closed under complement, kleene star, reversal and substitution.

---

(2) Construct a Non-Deterministic Finite Automata with $\varepsilon$-transitions to represent the regular expression $(a+b)^*(ab)^*$.

Ans:-

Given regular expression:

$$(a+b)^*(ab)^*$$

Step 1: Understand the regular expression.

It consists of two parts:

- $(a+b)^*$ : Represents zero or more occurences of either a or b.
- $(ab)^*$ : Represents zero or more occurences of the sequence ab.

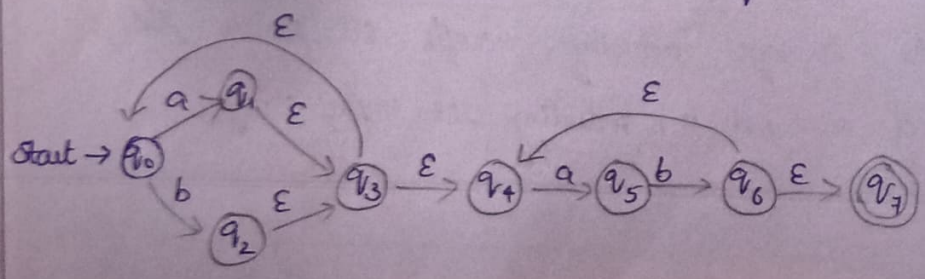
The complete expression represents any combination generated by these.

Step 2: Construct the $\varepsilon$-NFA

Using Thompson's construction, automata for:

1. $(a+b)^*$
2. $(ab)^*$

→ connect them using $\varepsilon$-transition.



## Step 3: Transition diagram

| State | Input | Next state |
|---|---|---|
| $q_0$ | a | $q_1$ |
| $q_0$ | b | $q_2$ |
| $q_3$ | $\varepsilon$ | $q_0$ |
| $q_1$ | $\varepsilon$ | $q_3$ |
| $q_2$ | $\varepsilon$ | $q_3$ |
| $q_3$ | $\varepsilon$ | $q_4$ |
| $q_4$ | a | $q_5$ |
| $q_5$ | b | $q_6$ |
| $q_5$ | $\varepsilon$ | $q_4$ |
| $q_6$ | $\varepsilon$ | $q_7$ |

## Step 4: Explanation

- The first part accepts any combination of a and b.
- $\varepsilon$-transitions allow the automation to repeat the first part or move to the second part.
- The second part accepts zero or more repetitions of ab.
- The final state $q_7$ indicates acceptance.

**Important observation:**

Since $(a+b)^* = \Sigma^*$ and $\varepsilon$ belongs to $(ab)^*$, the given expression is equivalent to:

$$(a+b)^* (ab)^* = (a+b)^*$$

Thus, the language actually accepts all possible strings consisting of a and b, including the empty string.

Give an informal description of the language denoted by the regular expression $(0+1)^*(00)$.

Given regular expression :

$(0+1)^*00$

Explanation:

The expression consists of two parts :

1. $(0+1)^*$ : Represents any combination of 0s and 1s, including the empty string.
2. 00 : Represents two consecutive zeros at the end of the string.

Therefore, the language contains all binary strings that end with two consecutive zeros.

Examples of accepted strings:

| Input string | Result |
|---|---|
| 00 | Accepted |
| 100 | Accepted |
| 000 | Accepted |
| 1100 | Accepted |
| 10100 | Accepted |
| 0100 | Accepted. |

Examples of rejected strings:

| Input string | Result |
|---|---|
| 0 | Rejected |
| 1 | Rejected |
| 01 | Rejected |
| 101 | Rejected |
| 1101 | Rejected |

Informal description:

The language consists of all strings over the binary alphabet $\Sigma = \{0,1\}$ that end with two consecutive zeros (00). The position before the final two zeros can contain any combination of zeros and ones, including an empty string.

④ Justify that the language L consisting of all strings with an equal number of a's and b's over the alphabet $\Sigma = \{a,b\}$ is not regular using pumping lemma.

Ans:

Given language:

$$L = \{ w \in \{a,b\}^* \mid n_a(w) = n_b(w) \}$$

Here:

- $n_a(w)$ represents the number of a's.
- $n_b(w)$ represents the number of b's.

We need to prove that L is not regular using the Pumping lemma.

Step 1: Assume the language is regular

According to the Pumping Lemma, if L is regular, there exists a pumping length p such that any string w belonging to L, where $|w| \geq p$, can be divided into three parts:

$$w = xyz$$

The following conditions must hold:

1. $|xy| \leq p$
2. $|y| > 0$
3. $xy^iz \in L$, for all $i \geq 0$.

Step 2: Choose a string

Consider the string: $w = a^p b^p$

That is,

$$w = aaa \ldots abbb \ldots b$$

This string contains:

- p number of a's.
- p number of b's.

Therefore,

$$w \in L.$$

**Step 3: Divide the string into xyz**

According to the Pumping Lemma:

$$w = xyz$$

Since, $|xy| \leq p$

both x and y must contain only a's because the first p symbols are all a's.

Also, $|y| > 0$

Therefore, y contains atleast one a.

Let, $y = a^k, \quad k > 0$

**Step 4: Apply pumping lemma (i=0)**

According to the Pumping Lemma:

$$xy^i z \in L$$

Since $y^i = \varepsilon$, the substring y is removed.

The resulting string becomes:

$$xz = a^{p-k} b^p$$

Now:

Number of a's $= p - k$

Number of b's $= p$

Therefore, $n_a(kl) \neq n_b(kl)$

This means the resulting string does not belong to L.

$xz \notin L$.

Step 5: Conclusion

This contradicts the Pumping Lemma condition that $xy^iz$ must belong to L for all $i \geq 0$.

Hence, our assumption that L is regular is false.

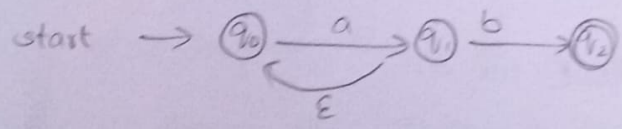
L is not regular language.

---

(5) Apply Arden's Theorem and derive the regular expression for the language represented by the given automata.

Ans:

The automata diagram required for Arden's theorem:




Arden's Theorem:-

If P and Q are regular expressions and ε is not present in P, then the equation:

$R = Q + RP$.

has the unique solution:

$$R = QP^*$$

Similarly, if:

$R = Q + PR$

then:

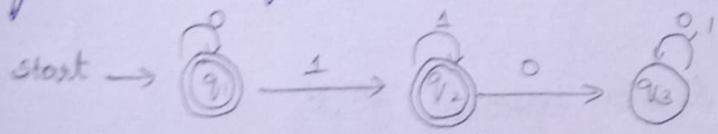
$$R = P^*Q$$

## Steps to solve using Arden's theorem:

1. Assign a variable to each state.
2. Write regular expression equations based on incoming transitions.
3. Identify the initial state and final state.
4. Apply Arden's theorem to solve the equations.
5. Substitute the obtained expressions to derive the final regular expression.

## State diagram:

Given state diagram:




**Step 1:** Form the equations

From the state transitions:

$$R_1 = \varepsilon + 0R_1 + 1R_2$$

$$R_2 = \varepsilon + 1R_2 + 0R_3$$

$$R_3 = 0R_3 + 1R_3$$

**Step 2:** Solve for $R_3$

$$R_3 = (0+1)R_3$$

Applying Arden's theorem:

$$R_3 = \emptyset$$

**Step 3:** Solve for $R_2$

$$R_2 = \varepsilon + 1R_2$$

Applying Arden's theorem:

$$R_2 = 1^*$$

Step 4: Solve for $R_1$

Substitute $R_2 = 1^*$ and $R_3 = \emptyset$:

$$R_1 = \varepsilon + 0R_1 + 11^*$$

Rearranging: $R_1 = 0R_1 + (\varepsilon + 11^*)$

Applying Arden's theorem:

$$R_1 = 0^* (\varepsilon + 11^*)$$

Since,

$$\varepsilon + 11^* = 1^*$$

Therefore,

$$\boxed{R_1 = 0^* 1^*}$$

Final Regular expression: $0^* 1^*$